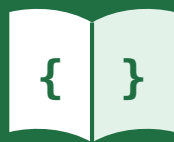

Module 18 — Spring Modulith

Exercises

Spring Boot Edu



2026-09-25

Contents

How to Work	2
Exercise 1 — Fix a Boundary Violation (Easy)	2
Exercise 2 — A New Module: Loyalty Points (Medium)	2
Exercise 3 — From a Call to an Event (Medium)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/18-modulith/exercise/`. Find the TODO comments.
2. The exercise application has the modules `catalog`, `review`, `order`, `inventory` and `loyalty`. The tests start PostgreSQL with Testcontainers, so Docker must be running.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/18-modulith/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/18-modulith/solution/`.

Exercise 1 — Fix a Boundary Violation (Easy)

Goal: the review module checks whether a book exists by using the catalog's repository directly. The repository is internal to the catalog module, and the module test fails.

Tasks (`review.ReviewService`):

- TODO 1 — use the catalog module's API instead of `catalog.internal.BookRepository`.

Hints:

- Run `Exercise1Test` first and read the message: Module 'review' depends on non-exposed type `...BookRepository` within module 'catalog'!
- Lesson section 3.1: the API is the module package. Look at what `catalog.CatalogService` offers.
- `Exercise1Test` uses `detectViolations().throwIfPresent()` instead of `verify()`. Lesson section 4 explains why.

Acceptance criteria: `Exercise1Test` and `Exercise1ReviewTest` pass.

Estimated time: 10 minutes

Exercise 2 — A New Module: Loyalty Points (Medium)

Goal: customers get one loyalty point for every full 10 of an order's total ($179.80 \rightarrow 17$ points). The loyalty module must work without the order module knowing it.

`loyalty.LoyaltyPoints` (the module's API with `pointsOf`, and `add` for the module itself) is given.

Tasks (package `loyalty`):

- TODO 2a — create a class that listens to the order module's `OrderPlaced` events.

- TODO 2b — compute the points and store them with `add(...)`.
- TODO 2c — declare in `package-info.java` that the module may depend on the order module only.

Hints:

- Lesson section 3.3: `@ApplicationModuleListener`. The listener class can be package-private.
- `total.divide(BigDecimal.TEN, 0, RoundingMode.DOWN).intValue()` rounds down.
- Lesson section 3.1: `@ApplicationModule(allowedDependencies = "order")` on the package, imported from `org.springframework.modulith`.
- The test is an `@ApplicationModuleTest`: only the loyalty module is started, and `Scenario.publish(...)` sends the event.

Acceptance criteria: all 4 tests in `Exercise2Test` pass.

Estimated time: 25 minutes

Exercise 3 — From a Call to an Event (Medium)

Goal: `OrderService.place` calls `inventory.reserve(...)` directly. So the order module depends on the inventory module, and it cannot be tested or changed without it. Turn the call into an event.

Tasks:

- TODO 3a (`order.OrderService`) — remove the call and the `Inventory` field; publish an `OrderPlaced` event instead (the record is given).
- TODO 3b (`inventory.StockReservations`) — reserve the stock when an `OrderPlaced` event arrives.
- TODO 3c (`inventory.Inventory`) — make `reserve` package-private again: now only the inventory module calls it.

Hints:

- Lesson section 3.2: inject `ApplicationEventPublisher` and call `publishEvent(...)` inside the `@Transactional` method.
- If you add the listener but forget to remove the call, `Exercise1Test` reports a cycle: `order → inventory → order`.
- `Exercise3OrderTest` starts only the order module. As long as `OrderService` needs an `Inventory` bean, the context cannot start.

Acceptance criteria: `Exercise3Test`, `Exercise3OrderTest` and `Exercise3InventoryTest` pass, and `Exercise1Test` stays green.

Estimated time: 25 minutes

Extra Challenge (Optional)

Add `spring-modulith-starter-jdbc` to the exercise project, create the `event_publication` table with a Flyway migration (as in the lesson), and write a test that shows a failed loyalty listener being resubmitted with `IncompleteEventPublications`.